Os **algoritmos de ordenação** (ou *sorting algorithms*) são métodos que organizam um conjunto de dados em uma determinada ordem — geralmente crescente ou decrescente. Eles são fundamentais em Ciência da Computação porque otimizam a busca, comparação e análise de informações.

A seguir **5 exemplos clássicos** 👇

🧩 1. Bubble Sort (Ordenação por bolha)

Ideia: compara pares de elementos adjacentes e os troca de lugar se estiverem na ordem errada.
Funcionamento:

1. Percorre o vetor várias vezes;
2. A cada passagem, o maior elemento "flutua" para o final.
   Exemplo:
   Vetor `[5, 3, 4, 1]` → compara 5 e 3 → troca → `[3, 5, 4, 1]` → segue até ordenar → `[1, 3, 4, 5]`.
   Complexidade: O(n^2).
   Uso: fácil de entender, bom para ensino, mas ineficiente para grandes conjuntos.

⚙️ 2. Insertion Sort (Ordenação por inserção)

Ideia: insere cada elemento na posição correta de uma lista parcialmente ordenada.
Funcionamento:

1. Começa no segundo elemento;
2. Compara com os anteriores e o coloca na posição certa.
   Exemplo:
   `[4, 2, 5, 1]` → pega 2 → insere antes do 4 → `[2, 4, 5, 1]` → continua até `[1, 2, 4, 5]`.
   Complexidade: O(n^2).
   Uso: eficiente para listas pequenas ou quase ordenadas.

🧮 3. Selection Sort (Ordenação por seleção)

Ideia: encontra o menor elemento e o coloca na primeira posição; depois repete o processo para o restante.
Funcionamento:

1. Percorre o vetor inteiro e encontra o menor valor;
2. Troca com o primeiro elemento;
3. Repete com os demais.
   Exemplo:
   `[7, 2, 4, 1]` → menor é 1 → troca com 7 → `[1, 2, 4, 7]`.
   Complexidade: O(n²).
   Uso: simples, mas pouco eficiente; útil para poucos elementos.

🚀 4. Merge Sort (Ordenação por intercalação)

Ideia: divide o vetor ao meio recursivamente e depois combina as partes ordenadas.
Funcionamento:

1. Divide até sobrar um elemento;
2. Junta os subvetores em ordem.
   Exemplo:
   `[5, 2, 8, 1]` → divide `[5, 2]` e `[8, 1]` → ordena cada um → intercala → `[1, 2, 5, 8]`.
   Complexidade: O(n log n).
   Uso: muito eficiente para grandes conjuntos; exige memória extra.

⚡ 5. Quick Sort (Ordenação rápida)

Ideia: escolhe um elemento chamado *pivô* e reorganiza o vetor de forma que os menores fiquem antes e os maiores depois.
Funcionamento:

1. Escolhe o pivô;
2. Coloca os menores à esquerda e os maiores à direita;
3. Repete o processo recursivamente.
   Exemplo:
   `[7, 2, 5, 1, 8]` → pivô = 5 → separa `[2,1]` e `[7,8]` → ordena → `[1,2,5,7,8]`.
   Complexidade: O(n log n) em média.
   Uso: é o algoritmo mais usado na prática por sua rapidez.

📊 Resumo comparativo

Algoritmo	Estratégia	Complexidade Média	Melhor uso
Bubble Sort	Comparação e troca	$O(n^2)$	Ensino básico, listas pequenas
Insertion Sort	Inserção ordenada	$O(n^2)$	Listas quase ordenadas
Selection Sort	Seleção mínima	$O(n^2)$	Simplicidade e pouco espaço usado
Merge Sort	Dividir e conquistar	O(n log n)	Grandes listas ordenadas estavelmente
Quick Sort	Dividir e conquistar	O(n log n)	Eficiência em grandes volumes

---

Abaixo estão os **cinco algoritmos de ordenação em linguagem C**, com **explicações passo a passo** em forma de comentários.
Cada exemplo é simples, didático e pronto para ser testado.

🧩 1. Bubble Sort (Ordenação por Bolha)

```c
#include <stdio.h>

void bubbleSort(int v[], int n) {
    int i, j, aux;
    // Percorre o vetor várias vezes
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            // Se o elemento atual for maior que o próximo, troca
            if (v[j] > v[j + 1]) {
                aux = v[j];
                v[j] = v[j + 1];
                v[j + 1] = aux;
            }
        }
    }
}

int main() {
    int v[] = {5, 3, 4, 1, 2};
    int n = 5;
    bubbleSort(v, n);

    printf("Vetor ordenado (Bubble Sort): ");
    for (int i = 0; i < n; i++) printf("%d ", v[i]);
    return 0;
}
```

🔹 **Explicação:**
Compara pares de elementos adjacentes e os troca se estiverem na ordem errada. O maior valor “sobe” ao final do vetor a cada passagem.

⚙️ 2. Insertion Sort (Ordenação por Inserção)

```
#include <stdio.h>

void insertionSort(int v[], int n) {
    int i, j, chave;
    // Começa a partir do segundo elemento
    for (i = 1; i < n; i++) {
        chave = v[i]; // Armazena o valor atual
        j = i - 1;
        // Move os elementos maiores para a direita
        while (j >= 0 && v[j] > chave) {
            v[j + 1] = v[j];
            j--;
        }
        // Insere a chave na posição correta
        v[j + 1] = chave;
    }
}

int main() {
    int v[] = {4, 2, 5, 1, 3};
    int n = 5;
    insertionSort(v, n);

    printf("Vetor ordenado (Insertion Sort): ");
    for (int i = 0; i < n; i++) printf("%d ", v[i]);
    return 0;
}
```

🔹 **Explicação:**
Cada elemento é comparado e “inserido” na posição correta de uma lista já ordenada.

🧮 3. Selection Sort (Ordenação por Seleção)

```c
#include <stdio.h>

void selectionSort(int v[], int n) {
    int i, j, min, aux;
    // Percorre o vetor
    for (i = 0; i < n - 1; i++) {
        min = i; // Assume que o menor é o atual
        // Procura o menor valor à frente
        for (j = i + 1; j < n; j++) {
            if (v[j] < v[min]) min = j;
        }
        // Troca se encontrou menor
        aux = v[i];
        v[i] = v[min];
        v[min] = aux;
    }
}

int main() {
    int v[] = {7, 2, 4, 1, 5};
    int n = 5;
    selectionSort(v, n);

    printf("Vetor ordenado (Selection Sort): ");
    for (int i = 0; i < n; i++) printf("%d ", v[i]);
    return 0;
}
```

🔹 **Explicação:**
Procura o menor elemento de cada passada e o coloca na posição correta.

🚀 4. Merge Sort (Ordenação por Intercalação)

```c
#include <stdio.h>

void merge(int v[], int inicio, int meio, int fim) {
    int i = inicio, j = meio + 1, k = 0;
    int aux[fim - inicio + 1];

    // Intercala os dois subvetores
    while (i <= meio && j <= fim) {
        if (v[i] <= v[j]) aux[k++] = v[i++];
        else aux[k++] = v[j++];
    }

    // Copia o que sobrou de cada lado
    while (i <= meio) aux[k++] = v[i++];
    while (j <= fim) aux[k++] = v[j++];

    // Retorna os valores ao vetor original
    for (i = inicio, k = 0; i <= fim; i++, k++)
        v[i] = aux[k];
}

void mergeSort(int v[], int inicio, int fim) {
    if (inicio < fim) {
        int meio = (inicio + fim) / 2;
        mergeSort(v, inicio, meio);     // Parte esquerda
        mergeSort(v, meio + 1, fim);    // Parte direita
        merge(v, inicio, meio, fim);    // Intercala
    }
}

int main() {
    int v[] = {5, 2, 8, 1, 3};
    int n = 5;
    mergeSort(v, 0, n - 1);

    printf("Vetor ordenado (Merge Sort): ");
    for (int i = 0; i < n; i++) printf("%d ", v[i]);
    return 0;
}
```

🔹 **Explicação:**
Divide o vetor ao meio e ordena as metades recursivamente, depois intercala as partes ordenadas.

⚡ 5. Quick Sort (Ordenação Rápida)

```c
#include <stdio.h>

void quickSort(int v[], int inicio, int fim) {
    int i = inicio, j = fim, pivo = v[(inicio + fim) / 2], aux;

    // Particionamento
    while (i <= j) {
        while (v[i] < pivo) i++;
        while (v[j] > pivo) j--;
        if (i <= j) {
            aux = v[i];
            v[i] = v[j];
            v[j] = aux;
            i++;
            j--;
        }
    }
    // Recursão nas partes esquerda e direita
    if (inicio < j) quickSort(v, inicio, j);
    if (i < fim) quickSort(v, i, fim);
}

int main() {
    int v[] = {7, 2, 5, 1, 8, 3};
    int n = 6;
    quickSort(v, 0, n - 1);

    printf("Vetor ordenado (Quick Sort): ");
    for (int i = 0; i < n; i++) printf("%d ", v[i]);
    return 0;
}
```

🔹 **Explicação:**
Escolhe um *pivô* e rearranja o vetor de forma que todos os menores fiquem à esquerda e os maiores à direita, aplicando o processo recursivamente.

📘 Resumo Final

Algoritmo	Tipo de Estratégia	Complexidade Média	Observações
Bubble Sort	Comparação e troca	$O(n^2)$	Simples, mas lento
Insertion Sort	Inserção ordenada	$O(n^2)$	Eficiente para listas pequenas
Selection Sort	Seleção mínima	$O(n^2)$	Poucas trocas, mas muitas comparações
Merge Sort	Dividir e conquistar	$O(n \log n)$	Estável e eficiente, mas usa memória extra
Quick Sort	Dividir e conquistar	$O(n \log n)$ (média)	Um dos mais rápidos na prática

Exemplos:

abaixo vocês tem **5 programas em C**, um para cada exercício, **com comentários explicativos linha a linha**. Cada programa lê os dados do teclado e aplica o algoritmo solicitado, depois exibe os resultados pedidos.

---

1) Insertion Sort — ordenar 10 notas (float) e mostrar menor e maior nota

```
#include <stdio.h>

int main() {
    int n = 10;                     // número de alunos (fixo: 10)
    float notas[10];                // vetor para armazenar as 10 notas
    int i, j;
    float chave;

    // Entrada: leitura das 10 notas
    printf("Digite 10 notas (separadas por Enter):\n");
    for (i = 0; i < n; i++) {
        scanf("%f", &notas[i]);     // lê cada nota e armazena no vetor
    }

    // Insertion Sort: ordena notas em ordem crescente
    for (i = 1; i < n; i++) {
        chave = notas[i];           // guarda o elemento a ser inserido
        j = i - 1;                  // posição anterior
        // desloca elementos maiores que 'chave' para a direita
        while (j >= 0 && notas[j] > chave) {
            notas[j + 1] = notas[j]; // move o elemento para a direita
            j = j - 1;               // avança para comparar com o
anterior
        }
        notas[j + 1] = chave;       // insere a chave na posição correta
    }

    // Após ordenação, o menor estará em notas[0] e o maior em notas[n-1]
    printf("\nNotas ordenadas (crescente):\n");
    for (i = 0; i < n; i++) {
        printf("%.2f ", notas[i]);  // exibe todas as notas ordenadas
    }

    printf("\nMenor nota: %.2f\n", notas[0]);       // menor nota
    printf("Maior nota: %.2f\n", notas[n - 1]);    // maior nota

    return 0; // encerra o programa
}
```

---

2) Selection Sort — ordenar 15 preços de produtos (float) e exibir média, menor e maior

```
#include <stdio.h>

int main() {
    int n = 15;                         // número de produtos
    float preco[15];                    // vetor para preços
    int i, j, min_index;
    float aux, soma = 0.0f;

    // Leitura dos preços
    printf("Digite os preços de 15 produtos (separados por Enter):\n");
    for (i = 0; i < n; i++) {
        scanf("%f", &preco[i]);         // lê cada preço
        soma += preco[i];               // acumula para cálculo da média
    }

    // Selection Sort: ordena preços do menor para o maior
    for (i = 0; i < n - 1; i++) {
        min_index = i;                  // assume o i-ésimo como o menor
        for (j = i + 1; j < n; j++) {
            if (preco[j] < preco[min_index]) {
                min_index = j;          // encontra índice do menor a partir
de i
            }
        }
        // troca preco[i] com preco[min_index] se necessário
        aux = preco[i];
        preco[i] = preco[min_index];
        preco[min_index] = aux;
    }

    // Exibe resultados: média, menor e maior preço
    printf("\nPrecos ordenados (crescente):\n");
    for (i = 0; i < n; i++) {
        printf("%.2f ", preco[i]);      // exibe vetor ordenado
    }
    printf("\nMedia dos precos: %.2f\n", soma / n);     // calcula média
    printf("Preco mais barato: %.2f\n", preco[0]);      // primeiro é o
menor
    printf("Preco mais caro: %.2f\n", preco[n - 1]);    // último é o maior

    return 0;
}
```

---

3) Bubble Sort — ordenar 8 velocidades (int) em ordem decrescente e imprimir ranking

```
#include <stdio.h>

int main() {
    int n = 8;                      // quantidade de veículos
    int vel[8];                     // vetor de velocidades (em km/h)
    int i, j, temp;

    // Leitura das velocidades
    printf("Digite as velocidades de 8 veiculos (km/h):\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &vel[i]);       // lê cada velocidade
    }

    // Bubble Sort modificado para ordem decrescente
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            // para decrescente, trocamos se o atual < próximo
            if (vel[j] < vel[j + 1]) {
                temp = vel[j];
                vel[j] = vel[j + 1];
                vel[j + 1] = temp;
            }
        }
    }

    // Exibe ranking do mais rápido ao mais lento
    printf("\nRanking de velocidades (decrescente):\n");
    for (i = 0; i < n; i++) {
        printf("%d°: %d km/h\n", i + 1, vel[i]); // mostra posição e
velocidade
    }

    return 0;
}
```

---

4) Merge Sort — ordenar 12 temperaturas mensais (float) e mostrar 3 menores e 3 maiores

```
#include <stdio.h>
#include <stdlib.h>

void merge(float v[], int inicio, int meio, int fim) {
    int n1 = meio - inicio + 1;            // tamanho da primeira metade
    int n2 = fim - meio;                   // tamanho da segunda metade
    float *L = (float*) malloc(n1 * sizeof(float)); // vetor temporário L
    float *R = (float*) malloc(n2 * sizeof(float)); // vetor temporário R
    int i, j, k;
    // copia elementos para L e R
    for (i = 0; i < n1; i++) L[i] = v[inicio + i];
    for (j = 0; j < n2; j++) R[j] = v[meio + 1 + j];
    i = 0; j = 0; k = inicio;
    // intercala enquanto houver elementos em ambos
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) v[k++] = L[i++]; // pega o menor de L ou R
        else v[k++] = R[j++];
    }
    // copia restantenes de L, se houver
    while (i < n1) v[k++] = L[i++];
    // copia restantenes de R, se houver
    while (j < n2) v[k++] = R[j++];
    free(L); free(R); // libera memória temporária
}

void mergeSort(float v[], int inicio, int fim) {
    if (inicio < fim) {
        int meio = (inicio + fim) / 2;    // calcula o ponto médio
        mergeSort(v, inicio, meio);       // ordena a metade esquerda
        mergeSort(v, meio + 1, fim);      // ordena a metade direita
        merge(v, inicio, meio, fim);      // intercala as duas metades
ordenadas
    }
}

int main() {
    int n = 12;                           // 12 meses
    float temp[12];                       // vetor para temperaturas
    int i;

    // Leitura das temperaturas mensais
    printf("Digite as 12 temperaturas medias mensais (em graus):\n");
    for (i = 0; i < n; i++) {
        scanf("%f", &temp[i]);
    }
    // Ordenação com Merge Sort (crescente)
    mergeSort(temp, 0, n - 1);

    // Exibe as três menores e as três maiores temperaturas
    printf("\n3 menores temperaturas:\n");
    for (i = 0; i < 3; i++) {
        printf("%.2f ", temp[i]);         // índices 0,1,2 -> menores
    }
    printf("\n3 maiores temperaturas:\n");
    for (i = n - 3; i < n; i++) {
        printf("%.2f ", temp[i]);       // índices n-3, n-2, n-1 -> maiores
    }
    printf("\n");
    return 0;
}
```

5) Quick Sort — ordenar 20 tempos de uso (int) em ordem decrescente; calcular média, maior e menor

```
#include <stdio.h>
// Função particiona o vetor em torno do pivô e retorna índice de partição
int partition(int v[], int low, int high) {
    int pivot = v[(low + high) / 2]; // escolhe pivô como elemento do meio
    int i = low, j = high, tmp;
    while (i <= j) {
    while (v[i]>pivot) i++;//para ordem decrescente, busca maior que pivô
        while (v[j] < pivot) j--;     // busca menor que pivô
        if (i <= j) {
            tmp = v[i]; v[i] = v[j]; v[j] = tmp; // troca elementos
            i++; j--;
        }
    }
    return i; // retorna índice de partição (posição onde direita começa)
}

// QuickSort recursivo para ordem decrescente
void quickSort(int v[], int low, int high) {
  if (low < high) {
   int pi = partition(v, low, high - 1); //particiona (note high-1 no uso)
        // chama recursivamente nas duas partições
        quickSort(v, low, pi - 1);
        quickSort(v, pi, high);
    }
}

int main() {
    int n = 20;                    // número de usuários
    int uso[20];                   // tempo de uso em minutos
    int i;
    long soma = 0;                 // soma para cálculo da média
    double media;
    int maior, menor;

    // Leitura dos tempos de uso
    printf("Digite os tempos de uso (minutos) de 20 usuarios:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &uso[i]);      // lê cada tempo
        soma += uso[i];            // acumula soma
    }

    // Ordena em ordem decrescente usando Quick Sort
    // Chamamos quickSort com intervalo 0 .. n-1
    quickSort(uso, 0, n - 1);

    // Calcula média e identifica maior/menor
    media = (double)soma / n;      // média de tempo de uso
    maior = uso[0];          // após ordenar decrescente, uso[0] é o maior
    menor = uso[n - 1];            // uso[n-1] é o menor

    // Exibe resultados
    printf("\nTempos de uso ordenados (decrescente):\n");
    for (i = 0; i < n; i++) {
        printf("%d ", uso[i]);     // exibe vetor ordenado
    }
    printf("\n\nTempo medio de uso: %.2f minutos\n", media);
    printf("Maior tempo de uso: %d minutos\n", maior);
    printf("Menor tempo de uso: %d minutos\n", menor);

    return 0;
}
```